Vulnerability Scanning Techniques

Introduction

Vulnerability scanning is a fundamental process in Vulnerability Assessment and Penetration Testing (VAPT) used to identify security weaknesses in computer systems, networks, and web applications. The goal of vulnerability scanning is to detect potential security flaws before attackers can exploit them.

Security professionals use automated scanning tools to analyse systems for open ports, outdated software, misconfigurations, and known vulnerabilities. These tools help organizations understand their security posture and prioritize remediation efforts.
In this lab, vulnerability scanning techniques are performed using tools such as Nmap for network scanning, Nikto for web server analysis, and OWASP ZAP for web application vulnerability assessment.

The results obtained from these scans are analysed and validated to reduce false positives and determine the severity of vulnerabilities using the Common Vulnerability Scoring System. This process helps classify risks as low, medium, high, or critical, allowing security teams to take appropriate remediation actions.

The objective of this exercise is to understand vulnerability scanning methodologies, perform practical scans using security tools, analyse detected vulnerabilities, and document the findings in a structured security assessment report.

Objective

The objective of this lab is to understand and perform vulnerability scanning techniques as part of the Vulnerability Assessment and Penetration Testing (VAPT) process. The exercise focuses on identifying potential security weaknesses in a system or web application using automated scanning tools and analyzing the results for risk assessment.

The main objectives include:

1. Perform Network Scanning

Use tools such as **Nmap** to discover open ports, running services, and service versions on the target system.

2. Conduct Web Vulnerability Scanning
   Identify security issues in web servers and applications using tools like **Nikto** and **OWASP ZAP**.

3. Identify and Analyze Vulnerabilities
   Detect common vulnerabilities such as outdated software, misconfigurations, and missing security controls.

4. Evaluate Risk Severity
   Classify identified vulnerabilities using the **Common Vulnerability Scoring System** to determine their severity levels (Low, Medium, High, or Critical).

5. Validate Scan Results
   Verify vulnerabilities manually to reduce false positives and ensure accurate findings.

6. Document Findings
   Record the scanning process, identified vulnerabilities, screenshots, and recommendations in a structured VAPT report.

Tools Used

Tool	Purpose
Nmap	Network scanning
Nikto	Web server vulnerability scanning
OWASP ZAP	Web application vulnerability scanning

Table -1 Tools Used

Methodology

The methodology followed in this lab is based on standard vulnerability assessment practices recommended by **OWASP** and **National Institute of Standards and Technology**

guidelines. The process includes several stages to systematically identify, analyse, and document security vulnerabilities.

1. Target Identification
   The first step is identifying the target system or web application for vulnerability scanning. This includes determining the IP address or domain name of the system to be assessed.

   Target IP: 192.168.1.105

2. Network Scanning
   Network scanning is performed to identify open ports, running services, and service versions on the target system. This helps determine potential entry points for attackers.

   Tool used: Nmap

   nmap -sV <target-ip>

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.1.105
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-16 02:41 -0400
Nmap scan report for 192.168.1.105
Host is up (0.00014s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
```

Figure -1 Network Scanning using Nmap

3. Web Server Vulnerability Scanning
   Web server scanning is performed to detect misconfigurations, outdated software, and common web server vulnerabilities.

   Tool used: Nikto

   nikto -h http://<target-ip>

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 192.168.1.105
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.1.105
+ Target Hostname:    192.168.1.105
+ Target Port:        80
+ Start Time:         2026-03-16 02:43:37 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the
/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily bru
://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the E
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp
```

Figure -2 Web Server Vulnerability Scanning using Nikto

Nikto identifies issues such as:

- Outdated web server versions
- Missing security headers
- Directory indexing vulnerabilities

4. Web Application Vulnerability Scanning

Automated web application scanning is performed to identify vulnerabilities such as cross-site scripting (XSS), SQL injection, and authentication weaknesses.

Tool used: OWASP-ZAP

Steps performed:

- Start the scanner
- Enter the target URL
- Run automated vulnerability scan
- Analyse detected vulnerabilities

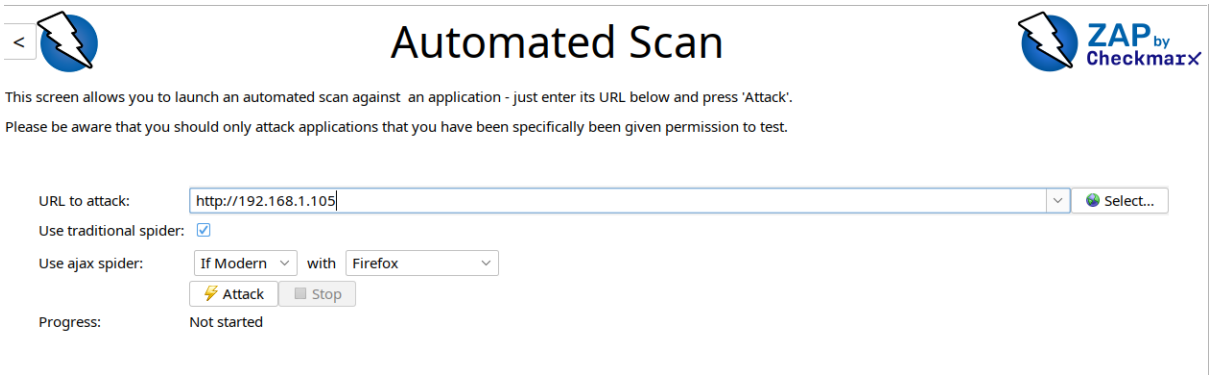

Figure -3 Web Application Vulnerability Scanning using OWASP-ZAP

5. Vulnerability Analysis
   The detected vulnerabilities are analysed to determine their potential impact and severity. The severity level of each vulnerability is assessed using the Common Vulnerability Scoring System.

   Severity classification

CVSS Score	Severity
0.0 – 3.9	Low
4.0 – 6.9	Medium
7.0 – 8.9	High
9.0 – 10	Critical

Table -2 Severity classification

6. Validation of Results
   To reduce false positives, identified vulnerabilities are manually verified using additional tools and manual inspection techniques.

   This step ensures that the reported vulnerabilities are valid and not incorrect scan results.

7. Documentation
   All findings, screenshots, scan results, and risk assessments are documented in a structured vulnerability assessment report. This documentation helps security teams understand the risks and implement appropriate mitigation strategies.

Learning Resources

1. OWASP Testing Guide
   The OWASP Testing Guide provides detailed methodologies for testing web application vulnerabilities.

   Topics covered:
   - SQL Injection testing
   - XSS testing
   - Authentication testing
   - Access control testing

2. NIST SP 800-115
   The National Institute of Standards and Technology document NIST SP 800-115 explains:
   - Vulnerability scanning methods
   - Security testing strategies
   - Risk assessment procedures

3. WannaCry Case Study
   The WannaCry exploited the CVE-2017-0144 vulnerability in Windows SMB protocol.

   Impact:
   - Over **200,000 systems infected**
   - Major organizations affected worldwide
   - Highlighted the importance of vulnerability scanning and patch management

---

Recommendations

Based on the vulnerabilities identified during the vulnerability assessment using **Nmap**, **Nikto**, and **OWASP ZAP**, the following security recommendations are suggested to mitigate the identified risks.

1. Update and Patch Software
   Outdated software components such as the web server and application libraries should be updated to the latest secure versions. Regular patch management helps prevent attackers from exploiting known vulnerabilities.

2. Disable Unnecessary Services
   Services such as Telnet, FTP, and other unused services should be disabled to reduce the attack surface. Only essential services should remain active on the server.

3. Implement Security Headers
   Security headers should be properly configured to protect against common web attacks.

   Recommended headers include:
   - X-Frame-Options

- Content-Security-Policy (CSP)
- X-Content-Type-Options
- Strict-Transport-Security (HSTS)

4. Restrict Directory Access
Directory listing should be disabled on the web server to prevent attackers from viewing sensitive files or directories.

5. Protect Sensitive Files
Sensitive files such as **phpinfo() pages**, configuration files, and test scripts should be removed or restricted to authorized users only.

6. Secure Cookies and Session Management
Cookies should be configured with proper security attributes such as:

- HttpOnly
- Secure
- SameSite

7. Implement Input Validation
All user inputs should be validated and sanitized to prevent attacks such as path traversal, SQL injection, and cross-site scripting (XSS).

8. Restrict Administrative Interfaces
Administrative tools such as database management panels should be accessible only to authorized users and protected by strong authentication mechanisms.

9. Regular Security Testing
Organizations should perform regular vulnerability scanning and penetration testing to identify new vulnerabilities and ensure the system remains secure.

10. Security Monitoring and Logging
Enable logging and monitoring mechanisms to detect suspicious activities and potential security incidents. Logs should be reviewed regularly to identify unusual behaviour.

Conclusion

The vulnerability assessment was conducted to identify security weaknesses in the target system using security scanning tools such as **Nmap**, **Nikto**, and **OWASP ZAP**. The assessment focused on discovering open services, web server misconfigurations, and application-level vulnerabilities.

The scanning results revealed several security issues including outdated software versions, missing security headers, directory listing, information disclosure, and potential web application vulnerabilities such as path traversal and weak session configurations. These vulnerabilities may allow attackers to gather sensitive information, exploit misconfigurations, or gain unauthorized access to the system.

The identified vulnerabilities were evaluated using the **Common Vulnerability Scoring System**, which helped categorize the risks into high, medium, and low severity levels. High-risk vulnerabilities require immediate remediation, while medium and low-risk issues should also be addressed to strengthen the overall security posture.

By implementing the recommended security controls such as updating software, disabling unnecessary services, configuring security headers, and restricting access to sensitive resources, the organization can significantly reduce potential attack vectors. Regular vulnerability assessments, security monitoring, and timely patch management are essential to maintaining a secure environment and protecting systems from evolving cybersecurity threats.